

Scenario3: LoRA Style Transfer and Advanced Inpainting Workflow

1. Introduction

In indie game development, keeping a consistent visual style is huge for building a recognizable identity and keeping players immersed. But while working on my own project, *Saving Money to Escape Poverty*, I quickly realized that drawing every asset by hand takes way too much time. It's also incredibly hard to keep the art style exactly the same over a long production cycle.

This is a massive headache for solo developers like me with limited resources. Thankfully, recent leaps in generative AI—specifically training Low-Rank Adaptations (LoRA) on latent diffusion models—offer a really promising workaround. LoRA lets you train a lightweight adaptation on top of a base model, meaning I can build a reusable, controllable style pipeline without needing a supercomputer.

This report tackles Scenario 3: LoRA Style Transfer and Advanced Inpainting Workflow, using my game as the testbed. The game is all about social mobility, so the environments need to visually evolve from Lv1 Poverty (makeshift and messy) up to Lv3 Affluence (luxurious and neat). This means my AI pipeline has to handle a wide variety of objects while strictly keeping the same aesthetic.

To pull this off, I trained a custom LoRA model named `smf_iso_style`. I judged it on two main things: (1) visual consistency (specifically dark lineart and flat cel-shading), and (2) geometric stability (keeping a strict 45-degree isometric cutaway perspective). Beyond just generating empty rooms, I also tested how the model handles object variations under different prompts to see if it actually understands what I'm asking for. Here is a quick roadmap of what I'll cover: Chapter 2 breaks down my dataset preparation, training config, and ComfyUI setup. Chapter 3 dives into the generated results, looking at style consistency and how the model handles different prompt complexities. Chapter 4 shows off an advanced inpainting workflow I built to refine assets. Chapter 5 goes over the model's limitations and my future plans. Finally, Chapter 6 wraps up my thoughts on whether LoRA is actually practical for game dev right now.

2. Methodology

This section walks through my entire pipeline, from getting the dataset ready to the actual LoRA training and ComfyUI workflow. I want to keep this process transparent and easy to reproduce.

2.1 Dataset Preparation

You can't train a good LoRA on bad data. For this project, I manually handpicked 28 isometric room illustrations from public sources and game art references, making sure they all fit a consistent anime/game asset style. The dataset covers three stages of the game (Lv1 Poverty, Lv2 Basic Living, Lv3 Affluence). This helps the model learn how visual content changes while the core art style stays the same.

My data cleaning process looked like this:

- Style filtering: Tossing out images with the wrong perspective or conflicting rendering styles.
- Noise removal: Manually erasing watermarks, UI text, arrows, and weird artifacts.
- Resolution normalization: Resizing most images to 768x512 (and a few to 512x768) to balance the composition and keep SD1.5 happy.

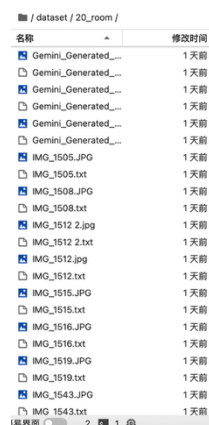


Figure 2-1: Organization of the Training Dataset showing Paired JPG and TXT Files.

2.2 Training Data Annotation Strategy

To make sure the AI properly linked my desired 45-degree flat-color look to specific prompts, I used a strict tagging format for all 28 training images. I paired each image with a plain-text .txt file containing a structured prompt.

- Trigger Word: I put `smf_iso_style` at the very beginning of every caption. This is the "magic key" to activate the trained LoRA weights.
- Tiered Content Tags: To support the "Poverty to Riches" narrative, I used tags like `lv1_poverty`, `lv2_modest`, and `lv3_luxury`. This categorized the room's complexity without messing with the stylistic backbone.
- Style Anchors: I slapped suffixes like `dark lineart` and `flat color` at the end of every prompt to hammer home that non-photorealistic game look.

Doing this manually was tedious, but it forced the model to decouple the content semantics (like furniture and lighting) from the visual style (like line weight and shading). This ended up being super important for the consistency I saw in my later tests.

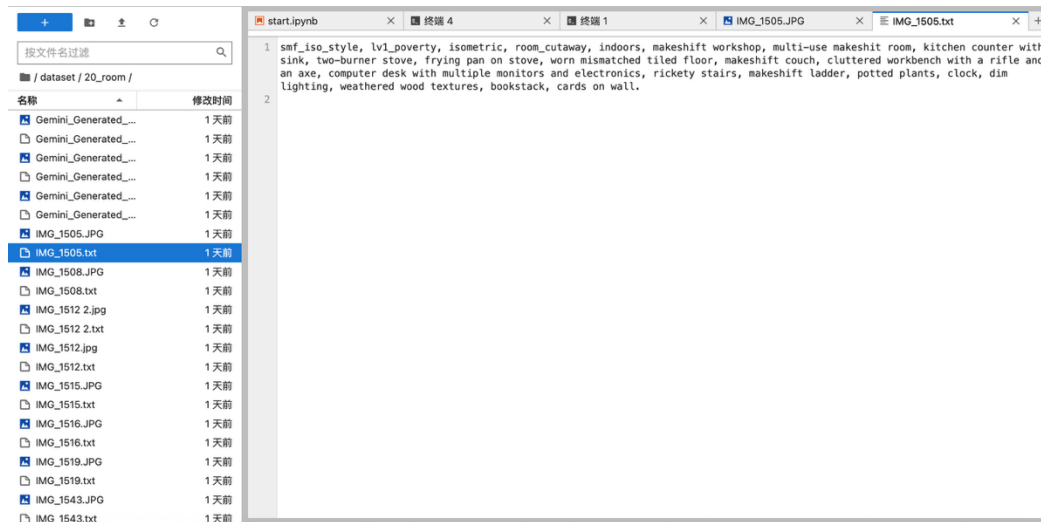


Figure 2-2: Example of a Structured Annotation File featuring the Trigger Word and Descriptive Tokens

2.3 LoRA Training Setup

I stuck with Stable Diffusion 1.5 (SD1.5) as my base model. Compared to newer stuff like SDXL, SD1.5 has a way more mature LoRA ecosystem and is much easier on the hardware, making it perfect for indie devs. I rented an AutoDL cloud server with an RTX 4090 to run the training.

- A. Batch Size and Training Steps: I went with a Batch Size of 4 and 1600 Max Steps. A larger batch size helps stabilize the training by reducing randomness in gradient updates. The 1600 steps gave the model enough time to figure out tricky structural patterns like the isometric perspective.

- B. Learning Rate Strategy: I split the learning rates to balance the visuals and the text understanding. The Unet got $1e-4$... Text Encoder was kept lower at $1e-5$ so I didn't fry the model's basic language skills. This stops it from overfitting to specific object arrangements.
- C. Learning Rate Scheduling: I used a 10% warmup paired with cosine decay. Warmup keeps things stable at the start, while cosine decay helps it finish smoothly in the later stages.
- D. LoRA Rank and Optimizer: A rank (Dim) of 16 was plenty for capturing a single style. I used AdamW8bit for the optimizer to save memory without losing performance.
- E. Resolution and Bucketing: I turned on Bucketing to handle the different aspect ratios in my dataset. This stops the trainer from aggressively cropping my rooms and helps preserve the geometric structure of the isometric scenes.

The screenshot shows the ComfyUI training interface with the following settings:

- Train batch size:** 4
- Epoch:** 12
- Max train epoch:** 12 (overrides max_train_steps, 0 = no override)
- Max train steps:** 1600 (overrides # training steps, 0 = no override)
- Save every N epochs:** 1
- Caption file extension:** .txt
- Seed:** 0 (Set to 0 to make random)
- Cache latents:** ☒ (Cache latents to disk is unchecked)
- LR Scheduler:** cosine
- Optimizer:** AdamW8bit
- Max grad norm:** 1
- LR scheduler extra arguments:** (Optional) eg: milestones=[1,10,30,50] gamma=0.1
- Optimizer extra arguments:** (Optional) eg: relative_step=True scale_parameter=True warmup_init=True
- Learning rate Unet:** 0.0001 (Set to 0 to not train the Unet)
- Learning rate TE:** 0.00001 (Set to 0 to not train the Text Encoder)
- LR warmup (% of total steps):** 10
- LR # cycles:** 1 (Number of restarts for cosine scheduler with restarts)
- LR power:** 1 (Polynomial power for polynomial scheduler)
- Max resolution:** 768,512
- Stop TE (% of total steps):** 0
- Enable buckets:** ☒
- Minimum bucket resolution:** 256 (Minimum size in pixel a)
- Maximum bucket resolution:** 2048 (Maximum size in pixel a)

Figure 2-3: Example of a Structured Annotation File featuring the Trigger Word and Descriptive Tokens

2.4 ComfyUI Workflow

All my actual image generation experiments happened in ComfyUI.

The main workflow is pretty standard:

1. Load the base model (I used AnythingV5 here).
2. Load the LoRA (smf_iso_style).
3. Encode the prompts (always including the trigger word).

4. Run it through the KSampler (steps = 25, CFG = 7.5).
5. Decode and save the image.

For the inpainting workflow, I had to add a few more nodes: an input image, a mask input, and an inpaint encoding node. This let me surgically edit specific parts of a room without affecting the rest of the image.

Workflow Modules

The pipeline breaks down into three parts:

1. Style Injection: Mixes the base model and the LoRA. I kept the LoRA strength between 0.6 and 0.8 to balance style and variety.
2. Inpainting Module: Lets me selectively fix things while ignoring the rest of the image.
3. Sampling and Rendering: Controls the final quality. For inpainting, getting the denoise parameter right is make-or-break. Too low, and nothing changes; too high, and it nukes the original image. I found 0.75–0.85 worked best.

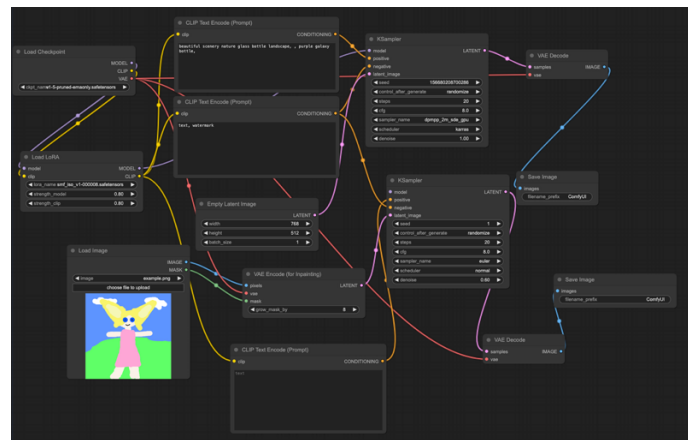


Figure 2-4: Standard ComfyUI Generation Workflow for smf_iso_style LoRA.

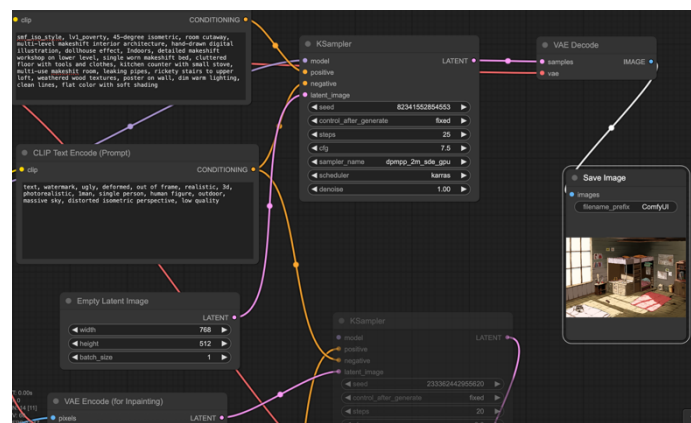


Figure 2-5: ComfyUI Image Output with Detailed Generation Parameters

2.5 Model Iteration & Checkpoint Selection

During the 12-epoch training, I saved checkpoints along the way to see how the style evolved. For my initial sensitivity tests, I used Epoch 8 because it felt "balanced" and let me clearly see how different weights affected the latent space. But for the final production pipeline and the advanced workflows (Experiments 2–5), I used the final Epoch 12. As the most baked-in version, it gave me the stylistic density I needed to keep those 45-degree angles stable across complex game scenes.

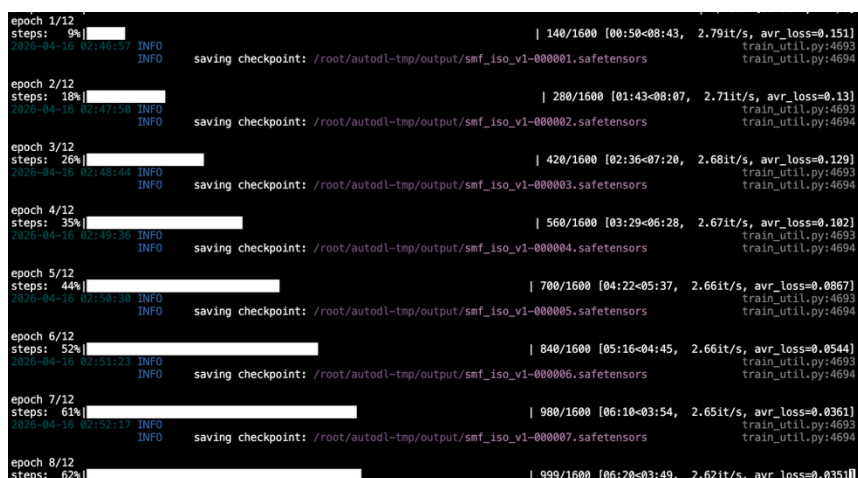


Figure 2-6: Real-time Training Progress and Loss Convergence Monitoring in the AutoDL Terminal.

3. Generated Images Analysis

3.1.1 Stylistic Sensitivity on Foundation Model (SD 1.5)

I first tested the LoRA on vanilla SD 1.5 to evaluate its raw stylistic extraction. At weight 0, the model had absolutely zero spatial awareness of isometric projections. When I bumped it to 0.5, the geometric structure started to show up, but it was noisy. At weight 0.8, my smf_iso_style fully clicked: the black lineart became continuous, and the colors shifted beautifully into that flat cel-shading look. Seeing this linear progression proved the LoRA successfully encoded the geometric and artistic rules from my training set.

Condition	LoRA weight=0	LoRA weight=0.5	LoRA weight=0.8
-----------	---------------	-----------------	-----------------

Output			
Figure No.	A1-1	A1-2	A1-3

Figure 3-1: Stylistic Sensitivity Analysis on SD 1.5 across different LoRA weights.

3.1.2 Cross-Model Robustness on AnythingV5

To see if the LoRA could generalize, I tried running it on AnythingV5. Interestingly, at weight 0, AnythingV5 already tried to draw a basic room layout, but with soft textures and complex lighting. When I cranked the LoRA up to 1.2, it successfully overrode the base model—the linework got thicker, and the shading simplified to match my game's style. Even though the visual shift wasn't as drastic as the SD 1.5 test (since AnythingV5 has a strong default look), the 45-degree perspective stayed totally rock-solid. This proved my LoRA's structural robustness.

Condition	LoRA weight=0	LoRA weight = 1.2
Output		
Figure No.	A2-1	A2-2

Figure 3-2: Cross-Model Robustness Test: AnythingV5 at Weight 0 vs. Weight 1.2.

3.1.3 Overfitting Diagnostics: Why 0.8 is the 'Sweet Spot'

By comparing weight 0.8 to 1.2, I found the overfitting threshold. At 1.2, the lines got "carbonized"—they became unnaturally thick and jagged, which ruined the look of smaller assets. The model also got super rigid and stopped listening to my lighting prompts. Because of this, I settled on a weight of 0.8 for actual production. It keeps the core stylistic “DNA” but leaves enough creative flexibility for the base model to do its job.

3.2.1 Observation of Stylistic Drift

Generating the Lv1 and Lv2 rooms worked great with high fidelity, but the Lv3

(Affluence) category was a disaster. The model completely lost the 45-degree isometric perspective and started spitting out wide-angle, painterly scenes.

Tier	Lv1 Poverty	Lv2 Modest	Lv3 Luxury
Output			
Figure No.	A3-1	A3-2	A3-3
Prompt	smf_iso_style, lv1_poverty, 45-degree isometric, room cutaway, multi-level makeshift interior architecture, dollhouse effect, Indoors, cramped makeshift dwelling, scavenged materials, oil drums, makeshift furniture, cluttered floor, leaky pipes, dirty floor, dim warm light, weathered wood texture, clean lines, flat color with soft shading cluttered floor, weathered wood textures, dim lighting, dark lineart, flat color	smf_iso_style, lv2_modest, 45-degree isometric, room cutaway, multi-level interior architecture, dollhouse effect, Indoors, clean stone cottage, cozy wooden furniture, basic bed, modular storage, working desk with computer monitor, indoor staircase, functional layout, posters and potted plants, bright warm light, clean lines, flat color with soft shading	smf_iso_style, lv3_luxury, 45-degree isometric, room cutaway, multi-level opulent interior architecture, hand-drawn digital illustration, dollhouse effect, Indoors, opulent luxury living space, grand chandelier, plush velvet sofa, ornate carved fireplace, extensive bookshelves, decorative rugs, leather armchair, massive bay window with scenic view, dim moody lighting, sophisticated layout, intricate woodwork, dark lineart, flat color with soft shading

Figure 3-3: Stylistic Consistency across Narrative Tiers (Lv1 Poverty, Lv2 Basic Living, and Lv3 Affluence)

3.2.2 Cause Analysis: Cross-Model Prior Competition

This happened because of a domain gap in the inference pipeline. I trained the LoRA on vanilla SD1.5, but I deployed it on AnythingV5 to get better rendering. The problem is that AnythingV5's default bias for "luxury" and "cinematic" is incredibly strong. In highly complex Lv3 scenes, the base model's internal aesthetic just bullied my lightweight LoRA and overrode its style constraints.

3.2.3 Cause Analysis: The 'Semantic Vacuum' of Sample Imbalance

On top of that, my dataset distribution made things worse. Out of 28 images, only 2 (about 7%) were luxury rooms. Because of this, the LoRA never really developed a robust semantic mapping for high-wealth interiors. It created a "semantic vacuum"—

when I typed "luxury", the LoRA didn't have enough weight to fight back, so AnythingV5 took the wheel. It was a tough lesson in making sure you have enough sample density for specific concepts.

3.2.4 Subtle Stylistic Migration between Lv.1 and Lv.2

Even though my Lv1 and Lv2 datasets were perfectly balanced (13 images each), I still noticed a subtle style shift. Lv1 felt gritty and hand-drawn, but Lv2 outputs looked a bit too digital and polished. I attribute this to two things:

- Latent Prior Alignment: Concepts like "dilapidated" in the Lv1 prompts naturally align with the rough, imperfect lineart weights the LoRA learned. But Lv2 prompts use words like "tidy" or "modern", which wakes up AnythingV5's love for cleaner gradients and sharp edges. So, the base model subtly pulls on the LoRA, smoothing out my intended hand-drawn texture.
- Object Density & Rendering Pressure: Lv2 rooms are packed with more structured assets like electronics, neat beds, and bookshelves. As the visual complexity increases, the base model's engine tries harder to add shading and materiality to keep things looking coherent. This extra detail watered down my flat color rule, sneaking in soft shadows that weren't there in the simpler Lv1 scenes.

3.3 Prompt Complexity Analysis

Complexity	Simple Prompts	Long Prompts
Output		
Figure No.	A4-1	A4-2
Prompt Text	smf_iso_style, lv1_poverty, 45-degree isometric, room cutaway, multi-level makeshift interior architecture, dollhouse effect, Indoors, makeshift workshop, dark lines, flat color	smf_iso_style, lv1_poverty, 45-degree isometric, room cutaway, multi-level makeshift interior architecture, hand-drawn digital illustration, dollhouse effect, Indoors, detailed makeshift workshop on lower level, single worn makeshift bed, cluttered floor with tools and clothes, kitchen counter with small stove, multi-use makeshift room, leaking pipes, rickety stairs to upper loft, weathered

		wood textures, poster on wall, dim warm lighting, clean lines, flat color with soft shading
--	--	---

Figure 3-4: Impact of Prompt Complexity on Generative Fidelity: Simple vs. Long Prompts.

3.3.1 Failure of Generalization in Simple Prompts

My tests showed that simple prompts just don't work to activate the core features of this LoRA. If I only type a few words, the output looks generic and completely misses the refined linework and isometric precision. I call this semantic under-specification: because I used highly detailed captions during training, the LoRA basically demands long, dense prompts to "unlock" those exact stylistic weights.

3.3.2 High Fidelity through Semantic Alignment in Complex Prompts

On the flip side, complex prompts gave me exceptional fidelity that perfectly mirrored the vibe of my training data. By writing prompts that mirrored how I tagged the training images—describing textures, lighting, and spatial clutter—I successfully triggered the high-quality features. A long prompt acts like a multi-dimensional trigger, making sure the LoRA's specific rules overpower the base model's defaults.

3.3.3 Technical Insight: Caption-Driven Feature Retrieval

This taught me a huge technical lesson: my `smf_iso_style` LoRA operates as a context-dependent model. The style isn't just tied to the trigger word; it's deeply interleaved with specific descriptive tokens. Having to write long prompts is a bit annoying, but it gives me superior granular control over the room's complexity, making it highly effective for my game.

There is a trade-off, though. While the aesthetics peaked with long prompts, the strict 45-degree angle sometimes slipped into a normal interior perspective. It seems like under a high semantic load, the model prioritizes drawing the objects over keeping the rigid geometric constraints.

3.3.4 Token-Level Translation Analysis: Successes vs. Semantic Casualties

To really see where the model breaks, I did a token-level audit on my long prompts to analyze how specific phrases translated into the image.

1. Successfully Translated Tokens (Successful Stylistic Alignment)

- Weathered wood textures & cluttered floor: These terms are well represented.

The model easily translated them into specific visual elements like worn

wooden flooring and scattered objects that match the training set. This shows the LoRA has strong semantic sensitivity to material details.

- Hand-drawn digital illustration & clean lines: Even though these are abstract, their effect is obvious. You can clearly see strong black outlines on beds and shelves, proving the LoRA successfully captured these tokens. It successfully suppressed AnythingV5's tendency to produce soft, gradient-based shading.

2. Ignored or Distorted Tokens (Semantic Casualties & Overrides)

- Room cutaway & 45-degree isometric: These are the most notable failures. Under heavy prompts, the model largely ignores them and generates a perspective-based scene with depth. It seems that when you list a ton of objects (bed, desk, stove), their inherent spatial biases override the LoRA's isometric rules.
- Flat color: Despite being explicitly specified, the images still had soft shading and lighting effects. In scenarios involving "poverty" environments, the model prioritizes atmospheric lighting to convey mood, effectively ignoring the flat color requirement.

3. Summary of Model Understanding Boundary This experiment highlights a key limitation. The LoRA performs strongly at the object-level aesthetic but struggles with global structural constraints. When prompts get content-heavy, it prioritizes object appearance and mood while sacrificing the 45-degree perspective. Geometric control is simply too vulnerable under complex prompts.

4. Advanced Workflow Showcase: Controllable Inpainting Pipeline

In real game dev, a basic text-to-image setup rarely gives you a perfect room in one pass. This chapter walks through an advanced inpainting pipeline I built in ComfyUI to supplement assets.

4.1 Technical Setup: Native Realignment

To make sure new objects blended into the background perfectly, I reverted to the native SD1.5 base model and used my Epoch 8 LoRA at 0.8 weight. Since the LoRA was trained on SD1.5, using the same base model minimizes latent drift. Using Epoch 8

instead of the final version gave the model a bit more flexibility, preventing the inpainting from looking overly rigid or overfitted. As a result, newly generated objects share the same style and blend better.

4.2 Operational Logic: Parallel Dual-Link Architecture

I set up a parallel dual-link workflow in ComfyUI to balance flexibility and GPU efficiency:

- On-demand activation: The workflow splits into a standard branch and an inpainting branch. I only trigger the inpainting module when I like the overall composition but need to make local corrections. This avoids redundant image generation.
- Human-in-the-loop control: I manually load a base image and paint a mask over the edit region. It's lightweight and intuitive. Although not fully automated, it guarantees every modification aligns with the game's logic.

Overview of the Parallel Dual-Link Inpainting Workflow in ComfyUI.	Masking for Targeted Asset Supplementation	Final Output
		

Figure 4-1: Advanced Inpainting Workflow with Parallel Dual-Link Architecture in ComfyUI.

4.3 Case Study: From Derelict Void to Functional Shelter

Here is a practical example of adding a bed into an empty Lv1 room.

4.3.1 Problem Diagnosis of the Original Image My initial generation nailed the 45-degree angle and the lineart style. However, the center of the room was a large empty space. For a room management game, such emptiness weakens the sense of habitation and ruins the expected player interaction.

4.3.2 Targeted Supplementation and Result Comparison I masked the empty floor and prompted for an old military cot, rusty metal bedframe, stained pillow. With denoise set

to 0.8, the system dropped a perfect little bed right into the mask. Comparing the two figures, the new bed perfectly matches the 45-degree grid and aligns with the lighting coming from the window on the left. This proves the inpainting process preserves both geometry and lighting. More importantly, it saves me from throwing away an otherwise high-quality image just because of a missing asset.

Before	After
	
A5-1	A5-2

Figure 4-2: Case Study: Adding a Functional Asset via Targeted Inpainting (Before vs. After).

4.4 Functional Highlights: Pipeline Versatility

This advanced workflow serves two main purposes for me:

- 1. Creative Asset Supplementation: As shown above, if a scene lacks a workbench or a safe, I can iteratively enrich the environment. I can inject prompts into specific regions without breaking the overall style. This heavily improves asset reuse.
- 2. Iterative Error Correction: Generated images always have glitches like floating objects or distorted shapes. Inpainting lets me regenerate just the problematic regions. This "localized correction" acts as a final quality control layer, which is essential for real game development.

Overall, this pipeline fully demonstrates the Replace/Transfer and Inpainting capabilities required in Scenario 3, providing a practical workflow for iterative asset refinement.

5. Limitations & Future Work

This chapter examines where the smf_iso_style LoRA completely falls apart and how I plan to fix it.

5.1 Failure Case Analysis: Boundary Testing

I threw some out-of-distribution scenarios at the model to see what would happen. It did not go well, especially for outdoor environments and human figures.

1. Outdoor Scene Failure: Collapse of Style and Perspective

- Observation: When asked for a street scene, the clean lineart style vanished. It looked like a heavily rendered mobile game illustration from the 2010s. The strict 45-degree perspective collapsed into a vague top-down view.
- Cause Analysis: First, inductive bias conflict. All 28 of my training images were indoor cutaway scenes. The model developed a strong bias for "closed spaces" and couldn't transfer the lineart logic to the outdoors. Second, base model dominance. Without strong guidance outdoors, AnythingV5 took over and overrode the learned features. It only kept the color palette while losing all structural constraints.

2. Human Figure Distortion: Semantic and Structural Conflict

- Observation: Asking it to put a person in the room created visually unstable results. Humans appeared as distorted pixel clusters with no anatomical coherence. Surrounding furniture also became warped.
- Cause Analysis: My dataset had zero humans, so the LoRA had no prior representation of organic structures. Plus, the LoRA's rigid geometric constraints (straight lines, right angles) directly conflict with the curved, flexible nature of human anatomy, leading to total instability.

Outdoor Street Environment	Indoor Scene with Characters
	
A6-1	A6-2

Figure 5-1: Boundary Testing Failure Cases: Stylistic Collapse in Outdoor Scenery and Anatomical Distortion in Human Figures.

5.2 Systematic Limitations and Root Causes

Based on my tests, here is a breakdown of the core issues:

Limitation	Root Cause
Style Drift	Dataset inconsistency. Even though I curated them, the 28 images still vary slightly in line thickness and saturation.
Geometric Instability	Weak spatial understanding. The model learned "angled lines" as a 2D pattern, not as a true 3D space.
Caption Dependency	Tag overfitting. The model demands long prompts instead of just using the trigger word, meaning style and content aren't fully separated.
Base Model Incompatibility	The domain gap between SD1.5 (training) and AnythingV5 (inference) forces me to crank up the LoRA weight too high.
Weight Saturation	Overfitting at later epochs. Pushing the weight to 1.2 makes outputs nearly identical, killing variability.

This domain gap taught me an important practical rule: For consistent results, you really need to use the LoRA with the exact same base model it was trained on.

5.3 Future Work and Outlook

5.3.1 Immediate Technical Improvements

- Dataset Expansion: I need to add outdoor streets and organic structures to improve generalization.
- Geometric Control with ControlNet: Text prompts aren't enough to force perspective. I need to integrate ControlNet with fixed depth maps to keep the geometry consistent.
- Hyperparameter Optimization: To fight overfitting, I'll explore lower learning rates, higher LoRA ranks, and dropout in future training.

5.3.2 Toward Modular and Scalable Pipelines

- **Multi-LoRA Combination:** Instead of one massive LoRA, I want to train separate, modular LoRAs for style, objects, and themes so I can mix and match them.
- **Automated Inpainting Workflow:** Right now, inpainting requires manual masking. I'd love to see agent-based automation handle this in the future based on scene context.

5.3.3 Reflection on Human-in-the-Loop Design The tighter you control the style, the less creative the AI gets. The best path forward is combining weaker, modular LoRAs with structured workflows like ControlNet and Inpainting. This gives you both consistency and flexibility.

Final Reflection on GenAI Tools To directly answer the assignment prompt about what's missing: current tools desperately lack native geometric control mechanisms, and they offer limited support for modular, reusable style systems. Fixing these issues would turn ComfyUI from an experimental tool into a real workhorse for asset production.

6. Conclusion

This study covers the full journey of building the `smf_iso_style` LoRA. While I got some great results under specific conditions, the harsh truth is that it's not production-ready for commercial game dev yet.

6.1 Summary of Effectiveness

Style transfer with just 28 images is technically feasible. But the usability is poor:

- **Stylistic Locality:** It only works in a very narrow context (indoor scenes). Add dynamic elements, and the style drifts.
- **Inability to Sustain a Worldview:** It can't generate the outdoors or characters, meaning it can't support a cohesive game world on its own.
- **Fragile Prompt Dependency:** This is the biggest bottleneck. Because of my detailed training captions, if I don't type a perfectly structured paragraph, the style breaks. That makes it a nightmare for anyone else to reuse.

6.2 The Production Gap: High Maintenance and Inconsistency

AI asset pipelines look efficient, but they are incredibly labor-intensive. Because it constantly messes up spatial logic or lines, I have to generate and filter a ton of outputs just to find one good one. By the time I prompt engineer, set up the workflow, and manually inpaint the mistakes, I've spent more time in the short term than if I just commissioned a professional artist.

6.3 Strategic Value: Prototype over Product

Right now, `smf_iso_style`'s true value is in prototyping. It provides low-cost visual placeholders early on, helping me quickly explore room layouts and atmospheres. It acts more like a dynamic moodboard for inspiration rather than a final asset printer.

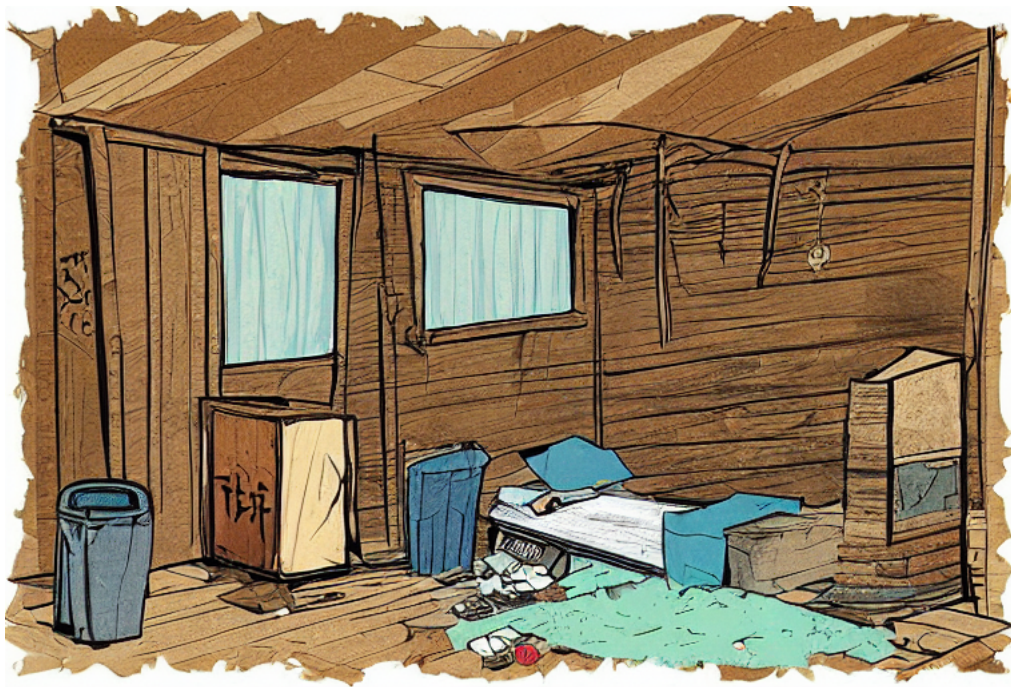
6.4 Final Verdict

At the end of the day, this LoRA hasn't bridged the gap from prototype to production tool. Small-scale training just isn't robust enough for complex game dev yet. AI isn't replacing artists anytime soon. Instead, we need to rely less on trial-and-error generation and combine it with strong structural tools like ControlNet. Until reliability improves, keeping a realistic, critical view of AI is much more practical than expecting full automation. This highlights exactly what's missing in today's tools: better geometric control and reusable workflows.

Appendix



A1-1



A1-2



A1-3



A2-1



A2-2



A3-1



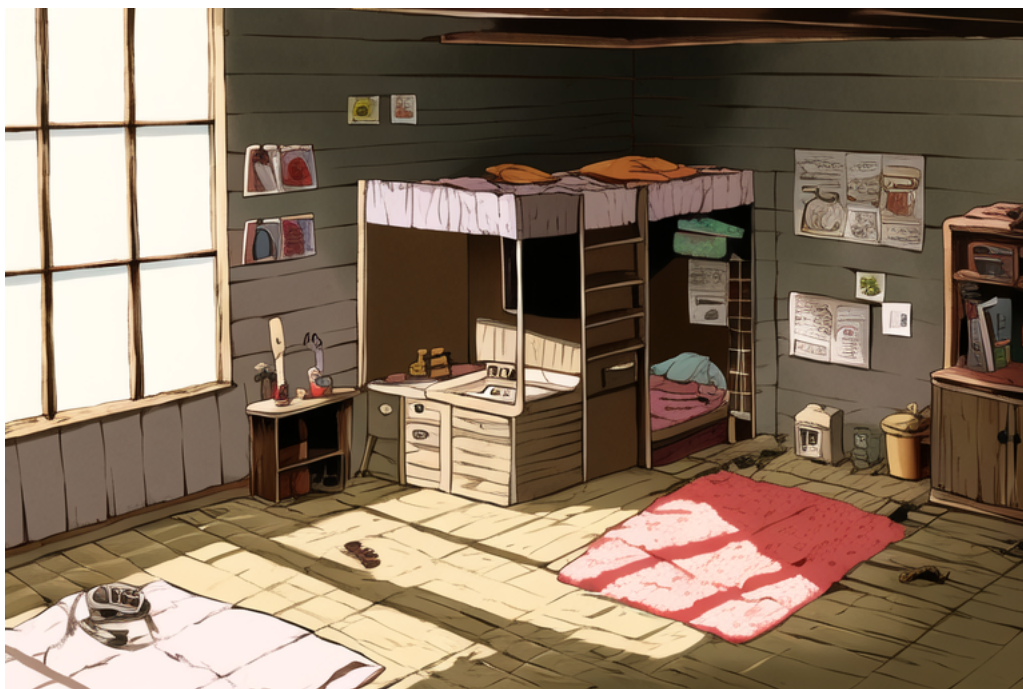
A3-2



A3-3



A4-1



A4-2



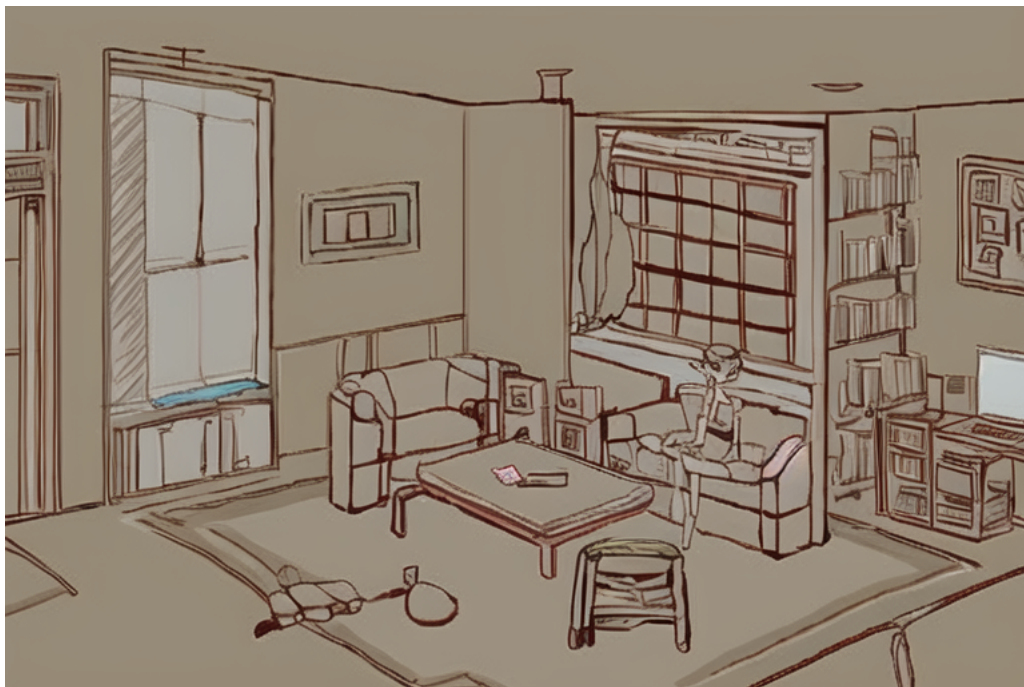
A5-1



A5-2



A6-1



A6-2



A7 dataset(9of28)